

CS223 Computer Architecture & Organization

Addressing Modes



John Jose

Associate Professor

Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

Addressing Modes

- ❖ The way in which the operand of an instruction is specified.
 - ❖ Implicit
 - ❖ Immediate
 - ❖ Direct
 - ❖ Indirect
 - ❖ Register
 - ❖ Register Indirect
 - ❖ Indexed
 - ❖ Auto increment/Auto decrement

Implicit Addressing

- ❖ Operand is implied, No need to specify the operand
- ❖ Instruction contain opcode only
- ❖ Eg 1: CMA (Complement Accumulator)
- ❖ Eg 2: SLA (Shift Left Accumulator)

Immediate Addressing

- ❖ Operand is part of instruction

- ❖ Operand = address field

- ❖ Eg: ADD #5



- ❖ Add 5 to contents of Accumulator

- ❖ 5 is the operand

- ❖ No memory reference for fetching operand. Hence it is fast

- ❖ Limited range

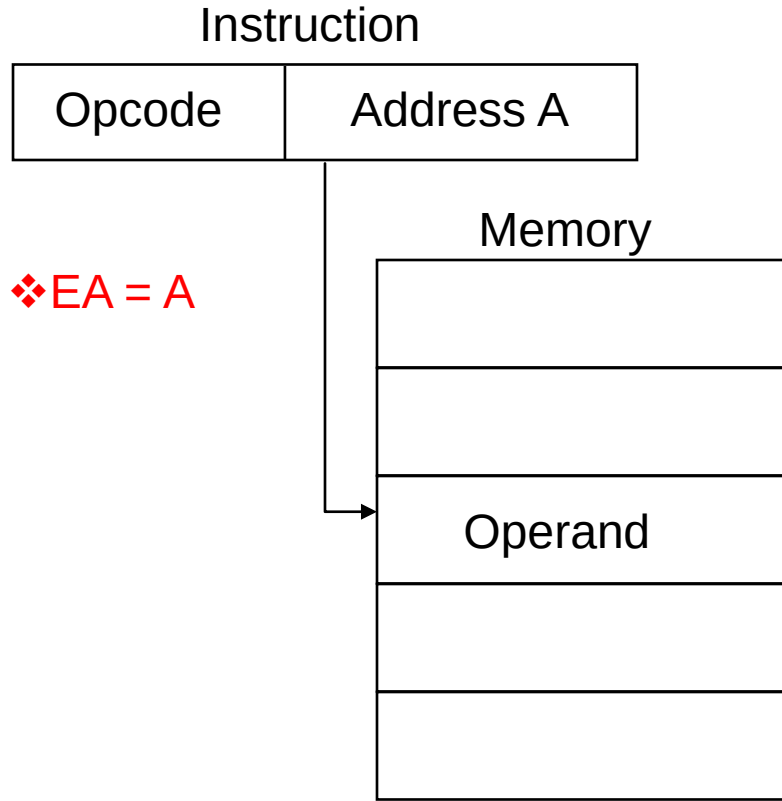
Direct Addressing

- ❖ Address field contains address of operand
- ❖ Eg: ADD 2000
 - ❖ Add contents of memory address 2000 to accumulator
 - ❖ Look in memory at address 2000 for operand
- ❖ Eg: LOAD R1, 5000
 - ❖ Load contents of memory address 5000 to R1
 - ❖ Look in memory at address 5000 for operand
- ❖ Single memory reference to access data
- ❖ Limited address space

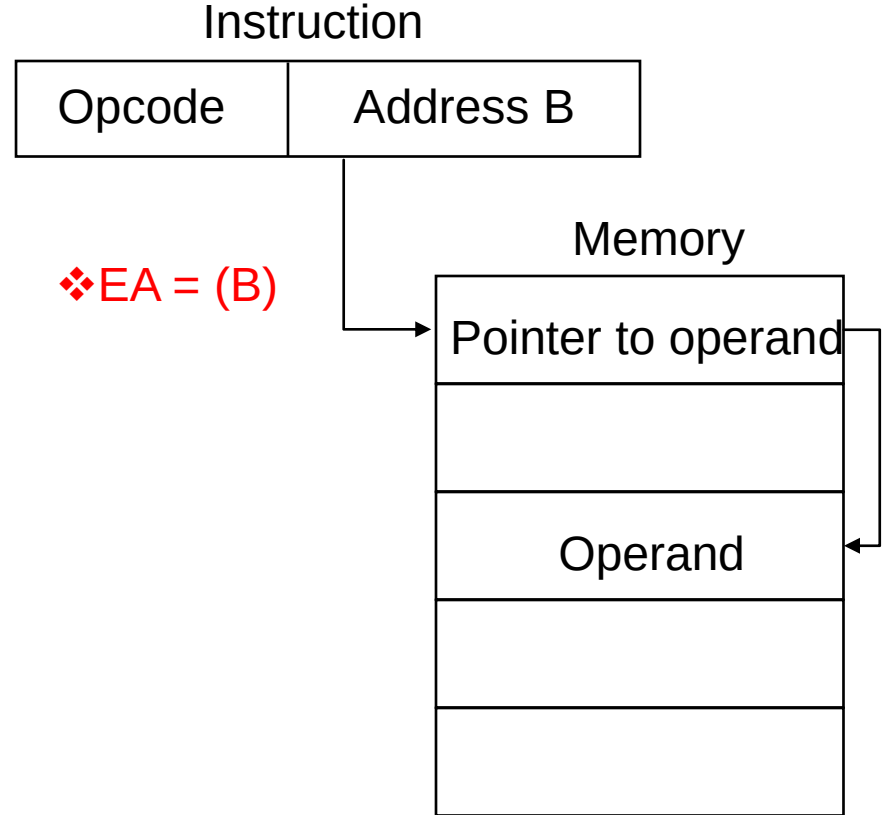
Indirect Addressing

- ❖ Memory cell pointed to by address field contains the address of the operand (indirection)
- ❖ $EA = (A) \rightarrow$ Look in A, find address (A) and look there for operand
- ❖ Eg: ADD (5000)
 - ❖ Add contents of memory pointed to by contents of 5000 to accumulator

Direct Addressing vs Indirect Addressing



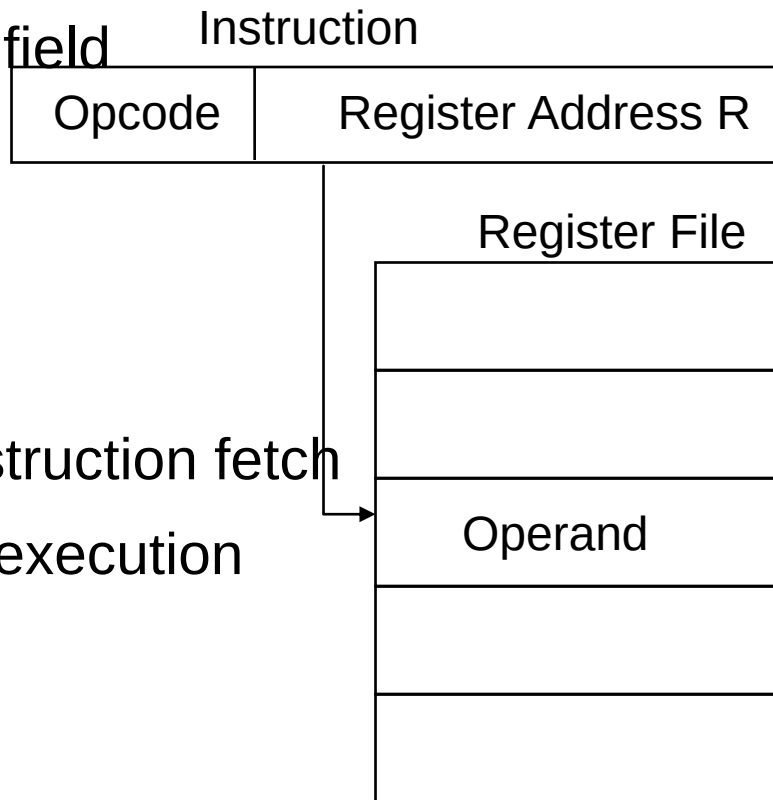
Direct Addressing



Indirect Addressing

Register Addressing

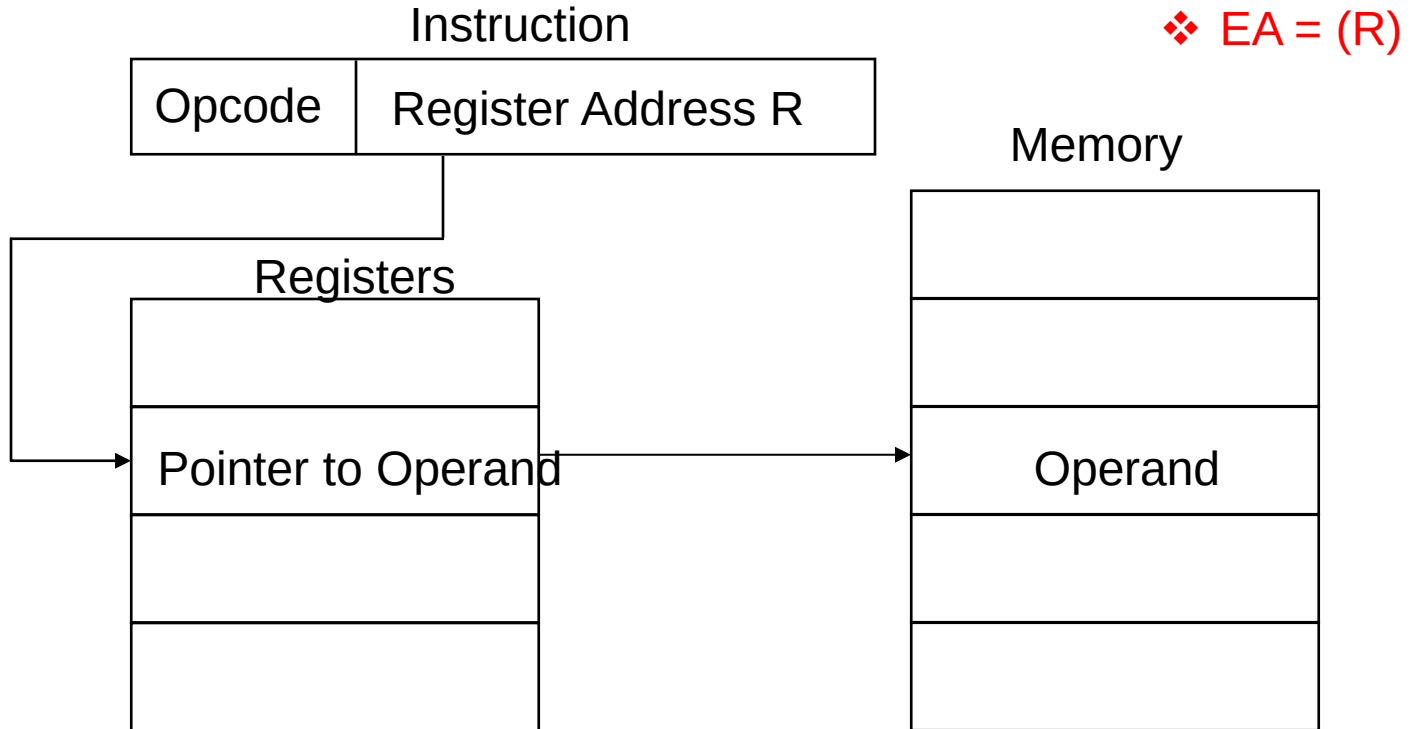
- ❖ Operand is held in the register.
- ❖ Its name is mentioned in address field
- ❖ $EA = R$ eg: ADD R1, R2, R3
- ❖ Limited number of registers
- ❖ Very small address field needed
 - ❖ Shorter instructions, Faster instruction fetch
 - ❖ No memory access, Very fast execution
 - ❖ Very limited address space



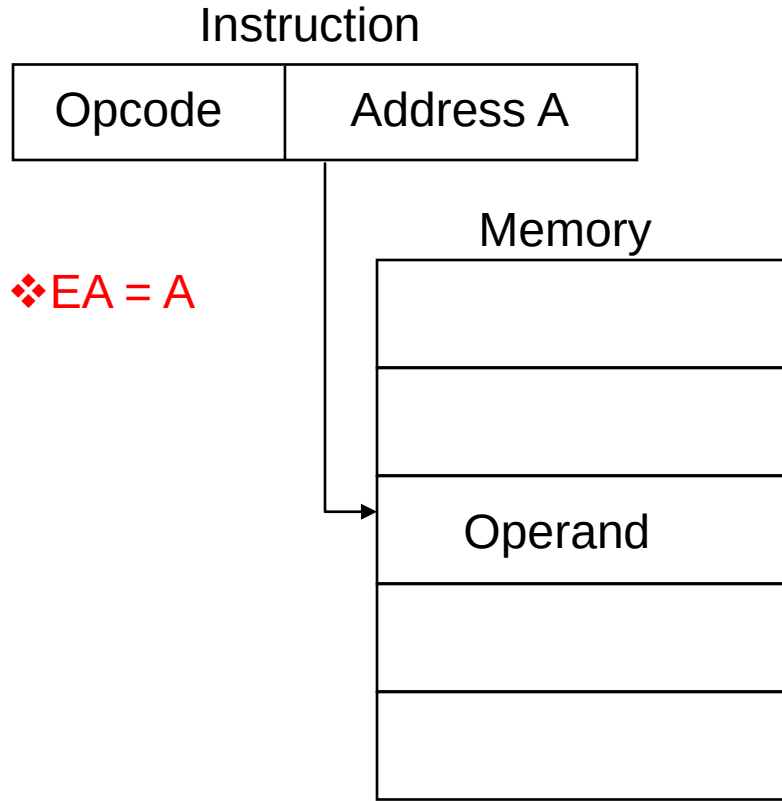
Register Indirect Addressing

- ❖ $EA = (R)$
- ❖ Operand is in memory cell pointed to by contents of register R
- ❖ Eg: `ADD R1, (R2)`
- ❖ Refer contents of R2, which is a memory address, contents of that memory location is the operand.
- ❖ Large address space (2^n)
- ❖ One fewer memory access than indirect addressing

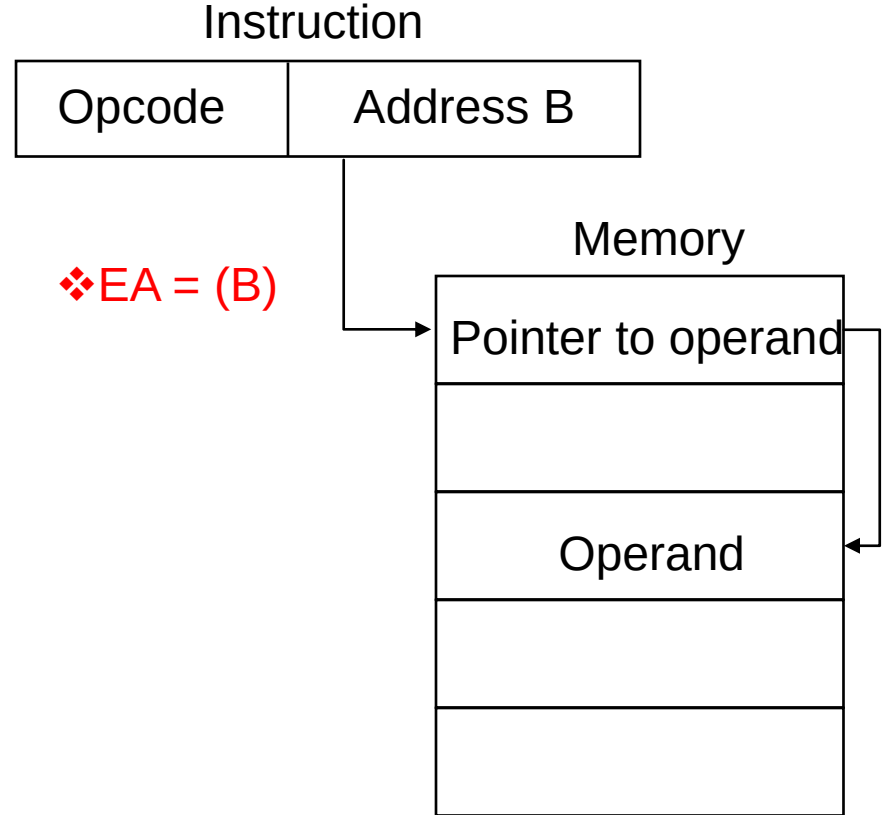
Register Indirect Addressing



Direct Addressing vs Indirect Addressing



Direct Addressing



Indirect Addressing

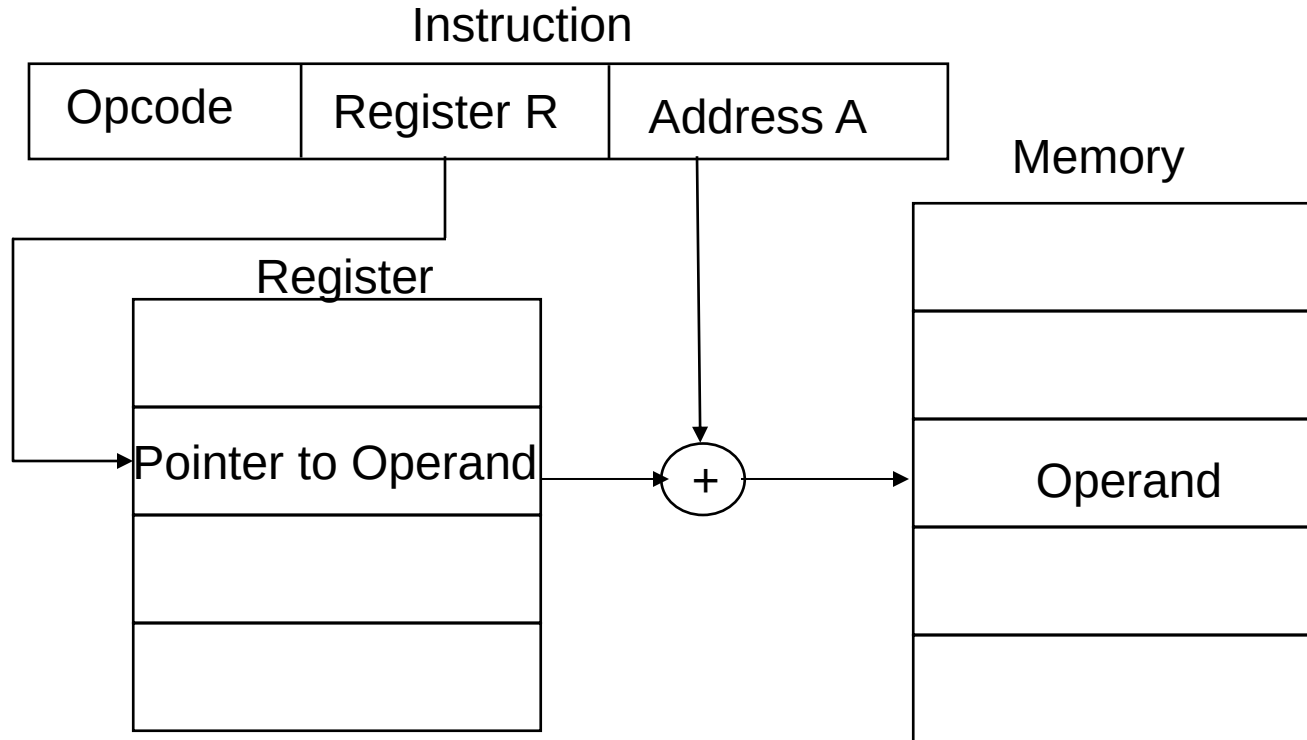
Displacement Addressing

- ❖ $EA = A + (R)$
- ❖ Address field is obtained by two values
 - ❖ $A = \text{base value}$
 - ❖ $R = \text{register that holds displacement}$
 - ❖ or vice versa
 - ❖ Eg: `LOAD R1, 16(R2)` $\rightarrow EA = [R2] + 16$

Displacement Addressing

❖ $EA = A + (R)$

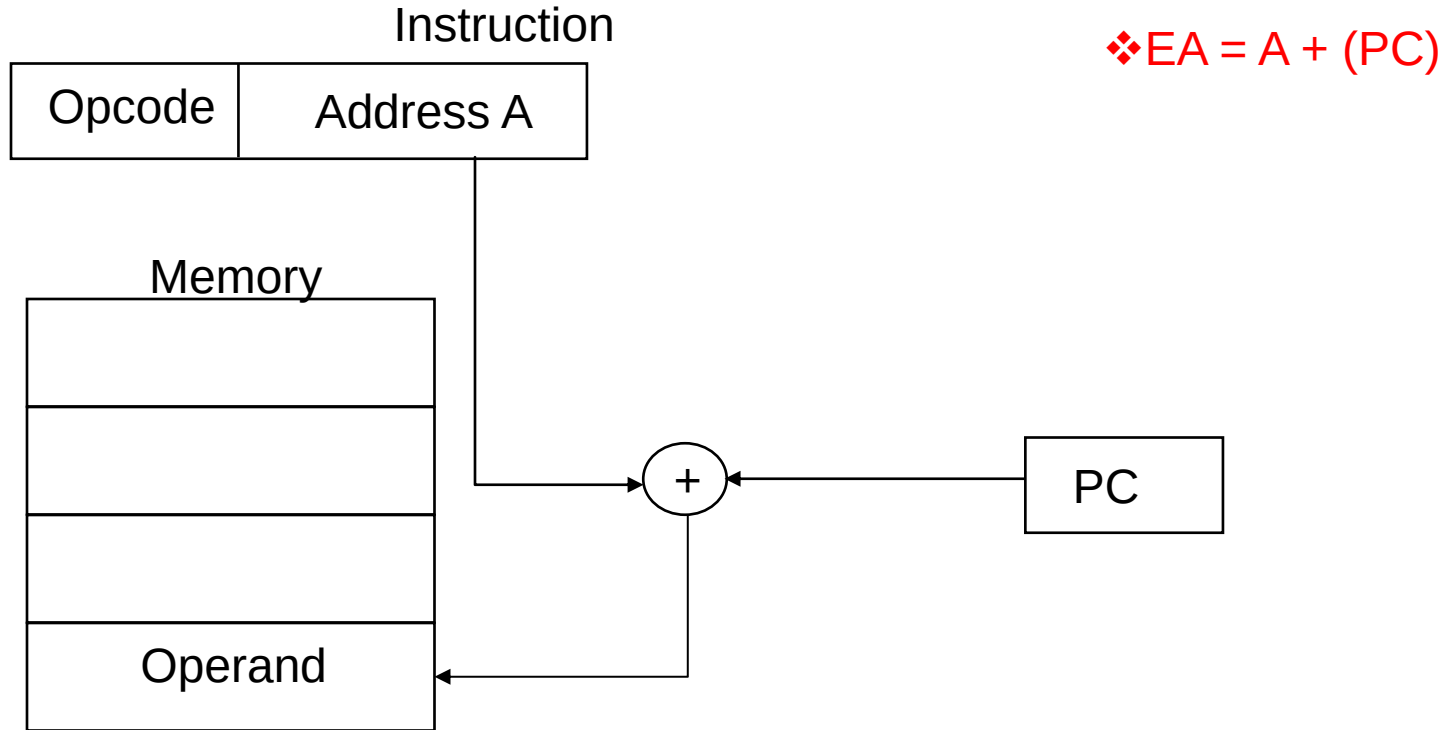
❖ `LOAD R1, 16(R2)`



Relative Addressing

- ❖ A version of displacement addressing
- ❖ $R = \text{Program counter, PC}$
- ❖ $EA = A + (PC)$
- ❖ Get operand from A location from current PC value [Jumps]
- ❖ Locality of reference

Relative Addressing



Base-Register & Indexed Addressing

❖ **Base-Register**

- ❖ A holds displacement
- ❖ R holds pointer to base address
- ❖ R may be explicit or implicit

❖ **Indexed Addressing**

- ❖ $A = \text{base}$
- ❖ $R = \text{displacement}$
- ❖ $EA = A + R$
- ❖ Good for accessing arrays
 - ❖ $EA = A + R$
 - ❖ $R++$

Auto increment and Auto decrement

- ❖ Operand is incremented or decremented after execution
- ❖ Eg 1: ADD R1, [R2]+ // $R1 \leftarrow R1 + M[R2]$, $R2++$
- ❖ Eg 2: ADD R1, -[R2] // $R2--$, $R1 \leftarrow R1 + M[R2]$



johnjose@iitg.ac.in
<http://www.iitg.ac.in/johnjose/>